DAY2: SYNTAX & STRUCTURE

Lexical Conventions:

Lexical conventions in Verilog are similar to C in the sense that it contains a stream of tokens.
Verilog is strictly **case-sensitive**
Every line must be terminated with a semicolon (;)

Whitespace: Characters such as spaces, tabs, newlines, and formfeeds are generally ignored by the compiler

Comments: You can use single-line comments starting with // for brief explanations, or multiple-line block comments starting with /* and ending with */ to disable larger sections of code during debugging.

Operators: Three main types of operators: unary, binary, and ternary

1. **Unary operators**: operate on a single operand and must appear to the left of that operand.
   For example, in the expression x = ~y;, the bitwise NOT (~) is a unary operator applied to y
2. **Binary operators** operate on two operands and must appear between them.
   For instance, in the expression x = y | z;, the bitwise OR (|) is a binary operator that evaluates y and z.
3. **Ternary** (Conditional) operators use two separate symbols (? and :) to separate three distinct operands.
   The syntax follows the format condition ? true_value : false_value.
   For example, in the expression x = (y > 5) ? w : z;, the first operand (y > 5) is evaluated as a condition
   If the condition is true, variable x is assigned the value of w, and if it is false, it is assigned the value of z.

Number: Treated as decimals by default
To represent numbers in other bases and define their exact bit-width, Verilog uses the strict syntax
[size]'[base_format][number]

The Standard Number Format:

- **Size**: This specifies the exact number of bits the value will occupy in hardware. It must always be written in decimal.

- **Base Format**: This indicates the radix of the number. You can use 'b or 'B for binary, 'd or 'D for decimal, 'h or 'H for hexadecimal, and 'o or 'O for octal

.

- **Number**: The actual value, written using valid digits for the chosen base. For example, hexadecimal numbers can use 0-9 and A-F, and both uppercase and lowercase letters are perfectly valid (e.g., 16'hCAFE is the same as 16'hcafe)

Readability: Use underscores (_) to separate digits. 32'hDEAD_BEEF is much easier to read than 32'hDEADBEEF

- **Unsized Numbers:** If you write a number without a base format, Verilog assumes it is a decimal.

If you write a number without a size specification (e.g., 'h1AD7), the simulator will automatically default to the host machine's standard width, which is typically 32 bits.

- **Negative and Signed Numbers:** To represent a negative number, the minus sign (-) must be placed before the size specification, not between the base format and the number.

For example: -6'd3 is correctly stored as the two's complement of 3
-6'sd9; // Signed decimal: 's' indicates signed arithmetic.

String: In Verilog, a string is a sequence of characters enclosed in double quotes (" ").
-Single-line restriction: Strings cannot be split or span across multiple lines.
-Whitespace preservation: Spaces and tabs inside a string are strictly preserved, "x + z" evaluates to exactly 5 characters.
-Memory and Sizing: To successfully store a string in a register, you must explicitly size the register as [8*N:1], where N is the total number of characters.
syntax reg [8*12:1] message = "Hello World!";

Identifiers: Names assigned to variables so that they can be referenced throughout your code.
Identifiers can consist of alphanumeric characters (a-z, A-Z, 0-9), underscores (_), and dollar signs ($).
Are strictly case-sensitive.
You cannot use hyphens in identifier names.
Must start with a letter or an underscore.
_internal and var23_g are valid, whereas 2var and $var_a invalid.
An identifier cannot be a reserved language keyword such as module or wire.

Keywords: Special, reserved identifiers used to define the built-in structural constructs of the language.
Strictly case-sensitive and must be written entirely in lowercase.
You cannot use keywords as identifiers to name your own variables, signals, or wires.

- A standard list of IEEE 1364 Verilog keywords includes:

always, and, assign, automatic, begin, buf, bufif0, bufif1, case, casex, casez, cell, cmos, config, deassign, default, defparam, design, disable, edge, else, end, endcase, endconfig,

endfunction, endgenerate, endmodule, endprimitive, endspecify, endtable, endtask, event, for, force, forever, fork, function, generate, genvar, highz0, highz1, if, ifnone, incdir, include, initial, inout, input, instance, integer, join, large, liblist, library, localparam, macromodule, medium, module, nand, negedge, nmos, nor, noshowcancelled, not, notif0, notif1, or, output, parameter, pmos, posedge, primitive, pull0, pull1, pulldown, pullup, rcmos, real, realtime, reg, release, repeat, rnmos, rpmos, rtran, rtranif0, rtranif1, scalared, showcancelled, signed, small, specify, specparam, strong0, strong1, supply0, supply1, table, task, time, tran, tranif0, tranif1, tri, tri0, tri1, triand, trior, trireg, unsigned, use, vectored, wait, wand, weak0, weak1, while, wire, wor, xnor, xor.

Verilog Module:

A Verilog module is the fundamental building block in Verilog HDL. It describes a hardware component (like a gate, flip-flop, ALU, or even a full processor) and defines its inputs, outputs, internal signals, and behavior.
It acts as a "black box" that encapsulates a specific piece of digital logic so it can be reused.
Both module and endmodule are foundational, reserved language keywords , case-sensitive and strictly lowercase.

ASPECT	MODULE CHARACTERISTICS	NOTES
Scope	All code must be within `module...endmodule`	No executable code allowed outside modules
Multiple Modules	One file can contain multiple modules	Defined in any order; simulator resolves dependencies
Port List	Optional (can be empty for testbenches)	Declared ports cannot be redeclared in module body
Reusability	Same module can be instantiated multiple times	Each instance has unique identifier
Hierarchy	Modules can contain sub-module instances	Forms tree structure with top-level root

A standard module typically includes:

- Basic Declaration: A module definition begins with the module keyword followed by a valid identifier that serves as the module's name and the entire block must be closed by the endmodule keyword.
- Port Declarations: The specific inputs, outputs, and bidirectional signals (inout) that allow the module to communicate with the rest of the circuit.
- Internal Data Types: Declarations of internal wire or reg signals used to connect logic exclusively within that module.
- Hardware Logic: The actual operations, defined using procedural blocks (like always), continuous assignments (assign), or by instantiating other, smaller sub-modules.

Example: D-ff with active low asynchronous reset

```
// Module called "dff" has 3 inputs and 1 output port
module dff ( input d,      // Data input
             input clk,    // Clock signal
```

```
            input rstn,    // Active-low asynchronous reset
           output reg q);      // Registered output
    // Sequential logic block: executes on every positive clock edge

    always @ (posedge clk) begin
        if (!rstn)       // Reset condition (active-low: rstn=0 triggers reset)
            q <= 0;      // Non-blocking assignment: reset output to 0
        else
            q <= d       // Non-blocking assignment: capture input data
    end
endmodule
```

The module dff represents a D flip flop which has three input ports d, clk, rstn and one output port q. Contents of the module describe how a D flip flop should behave for different combinations of inputs. Here, input d is always assigned to output q at positive edge of clock if rstn is high because it is an active low reset.

Top-level module: It is the highest module in the structural tree of a given hardware design. It represents the entire completed system or chip.
The inputs and outputs (ports) of a top-level module directly map to the physical input/output pins of the final hardware chip
It acts as the master wrapper that instantiates and connects all the lower-level sub-modules together to achieve the final system functionality.

Hierarchical names: Uses a dot (.) notation to define the exact path to the variable. The path traces through the instantiated names of the modules, not the module keywords themselves. For example, imagine a top-level testbench (tb_main) that instantiates a processor module (proc_inst). Inside the processor, there is an ALU module (alu_0) that contains a register named overflow_flag. The hierarchical name to access that specific flag from the testbench would be: tb_main.proc_inst.alu_0.overflow_flag
Hierarchical naming allows you to navigate that structural tree.
While highly encouraged for testing, using hierarchical names in your actual synthesizable hardware code (RTL) is generally considered a bad practice.

What is test bench?

A testbench is a specialized, non-synthesizable module used exclusively for simulation.
Its primary purpose is to verify the functional correctness of your hardware design—commonly referred to as the Device Under Test (DUT) or Unit Under Test (UUT)—before it is ever programmed onto a physical chip.
From a simulator perspective, testbench is the top level module.

The core characteristics of a Verilog testbench:

- No Input/Output Ports: A top-level testbench module typically has no ports (e.g., module tb_main;) because it is a closed simulation environment that provides its own stimuli and does not connect to any external hardware.
- Not Synthesizable: Because testbenches are not meant to be translated into physical hardware, you can freely use Verilog simulation constructs that have no physical equivalent, such as $display for printing text, or #10 for arbitrary time delays.

- Stimulus Generation: The testbench instantiates the DUT and uses initial or always blocks to drive inputs into it. This includes generating clock toggles, asserting resets, and feeding specific data patterns to see how the DUT responds.
- Hierarchical Peeking: As mentioned in our earlier discussion on hierarchical names, testbenches heavily utilize dot notation (e.g., tb_main.dut_inst.internal_wire) to monitor or force values onto internal signals hidden deep within the design. This allows you to verify internal logic without needing to route those signals out through the module's ports.

Syntax:

```
/* Testbench code - TOP-LEVEL MODULE for simulation
From simulation perspective, this is the root of the hierarchy
because 'design' module is instantiated within this module
*/
module testbench;  // No ports (testbench is self-contained)
// Instantiate the design-under-test (DUT)
        design d0 ( [port_list_connections] );     // Connect testbench signals to design ports

              // Testbench components:
              // - Clock generation (initial/always blocks)
             // - Stimulus generation (initial blocks with test vectors)
            // - Response checking (always blocks with assertions)
             // - Waveform dumping ($dumpfile, $dumpvars)
endmodule
 // Simulation hierarchy: testbench -> design -> {mod1, mod2} -> {mod3, mod3, mod4,
mod4}
```

Verilog Ports:

Ports are a set of signals that act as the inputs and outputs for a module.

Types of Ports Verilog provides three fundamental types of ports:

- input: Used exclusively to receive values from outside the module, such as clock signals, resets, or data inputs.
- output: Used exclusively to send values to the outside, such as computed results or status flags.
- inout: Bidirectional ports that can send or receive values.
  These are essential for memory interfaces (like DDR or SRAM) where data lines are shared. When using inout ports, it is critical to use tri-state logic (e.g., assign data = output_enable ? internal_data : 8'bz;) to ensure the port goes into a high-impedance state when not driving, preventing bus contention.

Port Type	Description	Use Cases
input	The design module can only **receive** values from outside using its `input` ports	Clock signals, data inputs, control signals, reset
output	The design module can only **send** values to the outside using its `output` ports	Computed results, status flags, data outputs
inout	The design module can either **send or receive** values using its `inout` ports (bidirectional)	Memory data buses, I2C/SPI data lines, tri-state buses

Syntax:

```
module my_design (
  input wire      clk,      // Clock input (explicit wire type)
  input           en,       // Enable input (implicit wire type)
  input           rw,       // Read/Write control (implicit wire)
  inout [15:0]    data,     // 16-bit bidirectional data bus
  output          int       // Interrupt output (implicit wire)
);
  // Design behavior as Verilog code
endmodule
```

By default, all ports are considered nets of type wire.
Procedural blocks require variables to be of type reg, you must explicitly declare the port as output reg to avoid compilation errors.

Signed vs. Unsigned Ports: Unless specified otherwise, ports are implicitly unsigned.
If your design performs arithmetic involving negative numbers (such as in Digital Signal Processing), you must explicitly use the signed attribute in the port declaration.

- Unsigned ports:

```
module my_module (
  input a,        // Unsigned by default
  input b,        // Unsigned by default
  output c        // Unsigned by default
);
  // ports a, b, and c are by default unsigned
endmodule
```

- Signed ports:

```
module signed_example (
  input signed a, b,      // Declare ports as signed in port list
  output c
);
  wire a, b;              // a, b inherit 'signed' from port declaration
  reg signed c;           // c is signed from reg declaration
endmodule
```

Mixing signed and unsigned operands in operations is highly discouraged, as Verilog will automatically default to an unsigned comparison, leading to unexpected logic bugs.

Port Declaration Styles :

- *Verilog-1995 (Non-ANSI):* In this older legacy style, only the port names are listed in the module header. The port directions (input, output) and data types are declared separately inside the module body.
  Example:

```
// Verilog-1995: Port list first, directions declared later
module test (a, b, c);              // List port names only
  input  [7:0] a;                   // inputs "a" and "b" are wires
  input  [7:0] b;
  output [7:0] c;                   // output "c" by default is a wire
  // Still, you can declare them again as wires to avoid confusion
  wire [7:0] a;
  wire [7:0] b;
  wire [7:0] c;
endmodule

module test (a, b, c);

  input  [7:0] a, b;                // Multiple inputs on one line
  output [7:0] c;                   // By default c is of type wire
  // port "c" is changed to a reg type (needed for always blocks)
  reg [7:0] c;
endmodule
```

- ***Verilog-2001 (ANSI-C Style):*** This is the standard for modern Verilog. It allows you to declare the direction, type, and bit-width all at once directly inside the port list.
  Example:

```
// Verilog-2001: Direction and type in port list (ANSI-C style)
module test (
  input [7:0]  a,                   // 8-bit input
               b,                   // "b" is also considered an 8-bit input (continues from
previous line)
  output [7:0] c                    // 8-bit output
);
  // Design content
endmodule


module test (
  input wire [7:0]  a,              // Explicit wire type (redundant, wire is default)
  input wire [7:0]  b,
  output reg [7:0]  c               // Output driven by reg (for use in always blocks)
);
  // Design content
endmodule
```